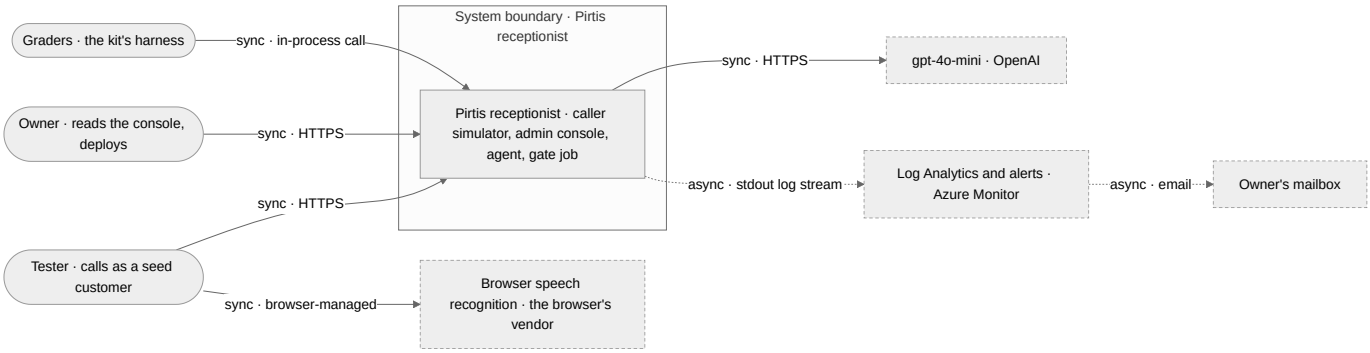
Architecture · printed from docs/architecture/containers.md, docs/architecture/data-flow.md, docs/architecture/sequences.md, docs/architecture/logic-flow.md, docs/architecture/deployment.md

> Verified against `3e57ec9` on 2026-10-31.

# Containers

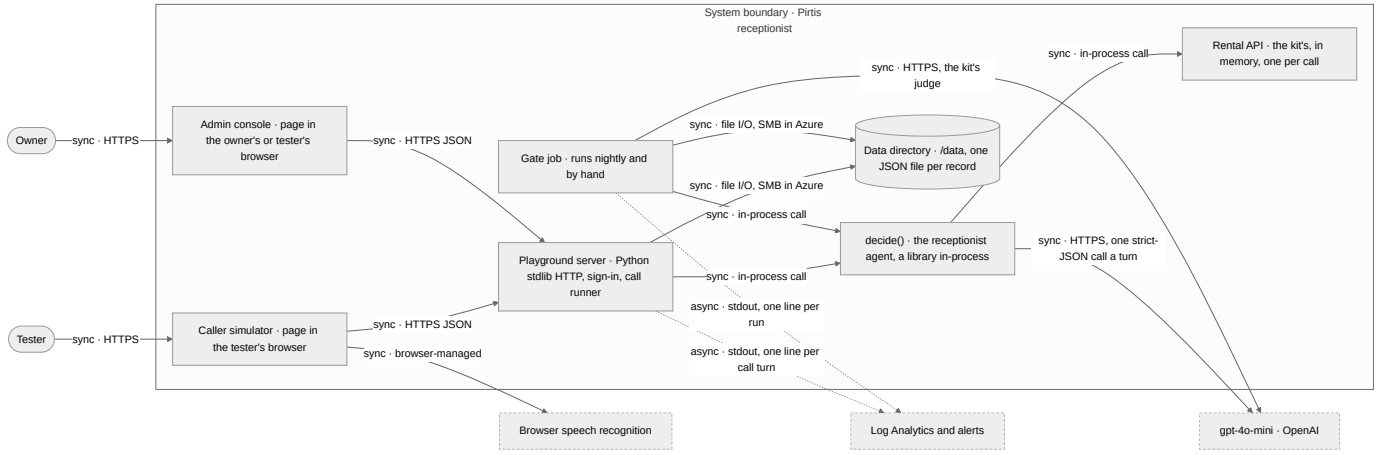
What runs, and what we don't control. Level 1 is the system as one box; level 2 opens it. Where it runs (resource group, registry, identity) is in `deployment.md`; who does what in order is in `sequences.md`; where personal data goes is in `data-flow.md`.

## Level 1 · context




The graders don't use the deployment: they run the kit's harness, which calls `decide()` in their own process. Everything else is the deployed system.

## Level 2 · containers




| Container | What it does | Evidence |
|---|---|---|
| Caller simulator | Pick a seed customer or an unknown number; type, or tap (or hold) to talk; the fogged booking card condenses as facts are heard; thumbs on every reply; scenario cards and the rating form | `python/pirtis/playground/static/caller.html`, `python/pirtis/playground/static/js/caller.js` |
| Admin console | Reliability (gate runs, the thermal view), Conversations, Traces, Feedback, Fleet, Human eval; all computed from content-free records except a stored call's trace | `python/pirtis/playground/static/admin.html`, `python/pirtis/playground/static/js/admin.js`, `python/pirtis/playground/analytics.py` |
| Playground server | The JSON API and the two pages; the sign-in (accounts we create, signed sessions); runs each call turn through `decide()` and the kit's handlers, saves the call after every turn | `python/pirtis/playground/server.py`, `python/pirtis/playground/auth.py`, `python/pirtis/playground/calls.py` |

| Container | What it does | Evidence |
|---|---|---|
| `decide()` | Turn gate (no model call), one strict-JSON understanding call, a deterministic policy that owns every rule and write, code-rendered speech, one telemetry line | `python/pirtis/agent/agent.py`, `python/pirtis/agent/turn.py`, `python/pirtis/agent/understand.py`, `python/pirtis/agent/policy.py`, `python/pirtis/agent/rules.py`, `python/pirtis/agent/speech.py` |
| Rental API | The kit's fixed API over an in-memory store, seeded per call from the fixtures' seed: units, availability, customers, bookings | `python/pirtis/api/memory.py`, `fixtures/seeds/base.json` |
| Gate job | The frozen gate in one process: the kit's Runner over the nightly suite, the extractor eval, a saved gate run, one summary line, pruning | `python/pirtis/ops/gate_job.py` |
| Data directory | Sessions, feedback, ratings, turn records, gate runs, calibrations; atomic writes, retention per kind | `python/pirtis/ops/store.py`, `python/pirtis/ops/model.py` |

## Outside the boundary

| Dependency | Vendor · region | Edge | If it is down | Fallback | Evidence |
|---|---|---|---|---|---|
| gpt-4o-mini | OpenAI API · [CONFIRM: processing region, US unless a data-residency project is set]. Azure OpenAI in an EU region instead via `LLM_BASE_URL` | sync · HTTPS | a turn that needs the model is put through to a person; an unreadable answer gets "could you say that again?"; the gate job fails with `llm_errors` above zero | turns the turn gate settles (still talking, silence, a request for a person) need no model; none for the rest | `python/pirtis/llm/client.py`, `python/pirtis/agent/agent.py` |
| Browser speech recognition | the browser's vendor (Chrome sends audio to Google's service; [CONFIRM: Safari and Edge]) · [CONFIRM: region] | sync · browser-managed | holding to talk does nothing useful | typing, always available | `python/pirtis/playground/static/js/caller.js` |
| Azure Files, hosting the data directory | Microsoft Azure · Germany West Central | sync · SMB | calls can't be saved or resumed; a gate run isn't saved | none | `deploy/terraform/main.tf` |
| Log Analytics, alerts, email | Microsoft Azure Monitor · Germany West Central | async · container stdout | no log queries and no alerts; calls go on | the admin console reads the data directory, not the logs | `deploy/terraform/monitoring.tf` |
| Container registry | Azure Container Registry · Germany West Central | sync · HTTPS image pull | a new revision or a job run can't start; the running app is unaffected | none | `deploy/terraform/main.tf` |

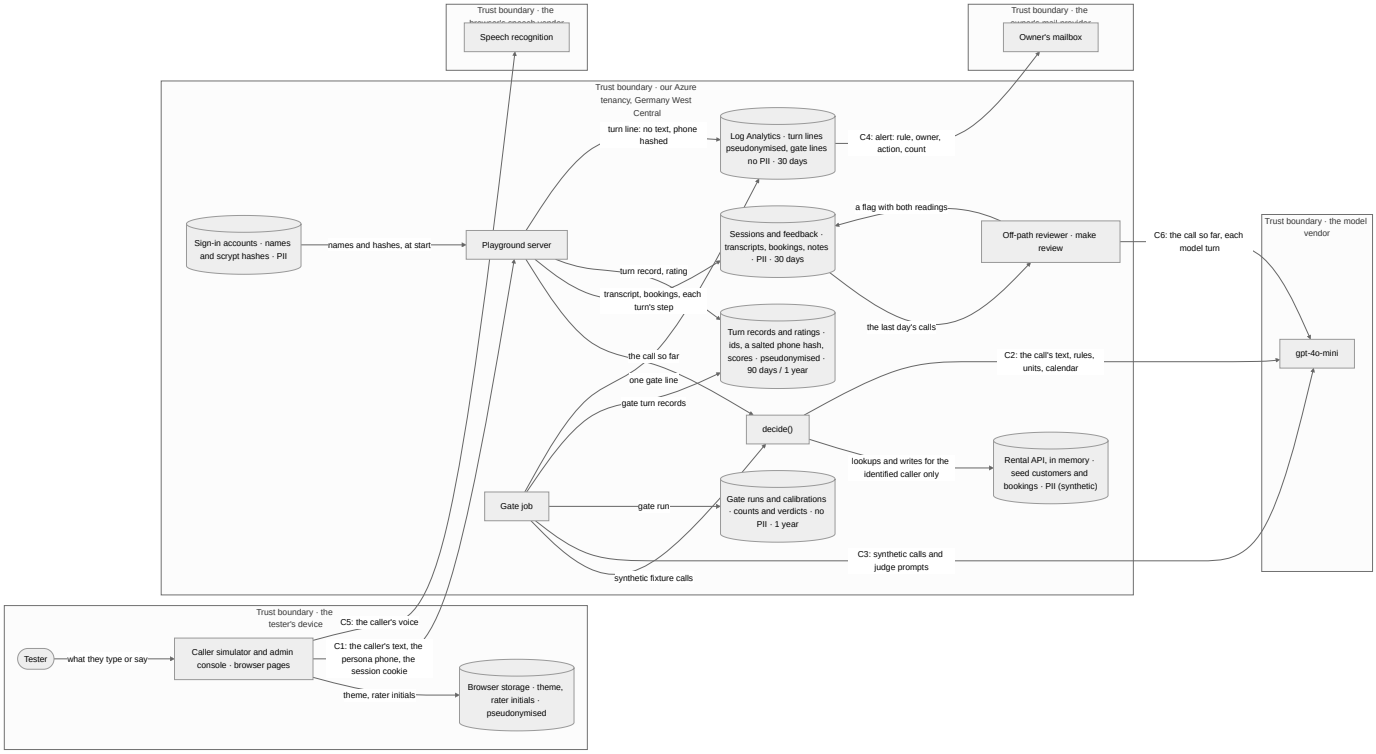
**Not built:**

- real telephony; the caller simulator stands in for the phone line
- streaming speech; a turn is one request
- a queue; every call turn is synchronous
- single sign-on; accounts are made by hand

> Verified against `3e57ec9` on 2026-10-31.

# Data flow

Where personal data sits, and what happens to it just before it leaves. The customers are made up (the fixtures' seed), and testers call as those customers. A tester's own words and voice are still personal data, so they are labelled as such.




## Stores

| Store | Label | Kept | Evidence |
|---|---|---|---|
| Sessions | PII: what the tester typed or said, the bookings the call made, each turn's step | 30 days | `python/pirtis/ops/model.py`, `python/pirtis/playground/calls.py` |
| Feedback | PII: a transcript snapshot and the rater's note | 30 days | `python/pirtis/ops/model.py` |
| Turn records | pseudonymised: ids, a salted hash of the phone's digits, counts; no text | 90 days | `python/pirtis/agent/telemetry.py`, `python/pirtis/ops/model.py` |
| Human ratings | pseudonymised: the rater's initials, a session id, scores | 1 year | `python/pirtis/ops/model.py` |
| Gate runs, calibrations | no PII: fixture ids, verdicts, counts | 1 year | `python/pirtis/ops/model.py` |
| Sign-in accounts | PII: account names and scrypt hashes, a Container App secret | until removed | `python/pirtis/playground/auth.py`, `deploy/Makefile` |
| Rental API | PII in shape, synthetic in content: the seed's customers, phones and bookings; rebuilt per call, never saved | the call | `python/pirtis/api/memory.py`, `fixtures/seeds/base.json` |
| Log Analytics | pseudonymised: turn lines (phone hash, ids, no text); gate lines and access lines (method, path without its query, status) hold no personal data | 30 days | `python/pirtis/agent/telemetry.py`, `python/pirtis/playground/server.py`, `deploy/terraform/main.tf` |
| Browser storage | pseudonymised: the theme and the rater's initials, on the tester's own device | until cleared | `python/pirtis/playground/static/js/caller.js` |

Retention is applied by the gate job's prune on every run (`python/pirtis/ops/store.py`).

## Boundary crossings

| ID | From → To | Data | Before it crosses | Region | Transfer mechanism (GDPR Ch. V) | Retention | Evidence |
|---|---|---|---|---|---|---|---|
| C1 | caller simulator → playground server (and the replies back) | the caller's text (speech already turned to text in the browser), the persona's phone, the session cookie; back: our replies, the booking card, a stored call's trace for the console | TLS only (HTTPS, HSTS); a signed-in session is required; spending is capped per account and per day | Germany West Central (EU) | none needed (the receiving side is in the EEA) | sessions 30 days | `python/pirtis/playground/server.py`, `python/pirtis/playground/auth.py` |
| C2 | `decide()` → model vendor | the call so far as text: caller lines, our lines, short notes ("caller identified", the caller's own bookings by sauna and dates); the rules; the units; a 42-day calendar | nothing is redacted: the caller's words go as said, a name included if they spoke it. Never sent: the phone number, other customers' data, past bookings' site and car details | [CONFIRM: OpenAI's processing region; the US unless a data-residency project is configured] | [CONFIRM: OpenAI's DPA with SCCs, Art. 46(2)(c)]; none needed on Azure OpenAI in an EU region | [CONFIRM: OpenAI API retention; abuse monitoring up to 30 days unless zero retention is agreed] | `python/pirtis/agent/understand.py`, `python/pirtis/llm/client.py` |
| C3 | gate job → model vendor | synthetic fixture calls, and the kit's judge prompts with our replies | nothing: synthetic data only | as C2 | as C2 | as C2 | `python/pirtis/ops/gate_job.py`, `python/pirtis/harness/judge.py` |
| C4 | Azure Monitor → owner's mailbox | an alert: the rule's name, its owner-and-action text, a count, a link; the queries project only versions, case ids and counts | nothing: the rules read content-free lines | [CONFIRM: the mail provider's region] | [CONFIRM: depends on the mail provider] | [CONFIRM: the mailbox's retention] | `deploy/terraform/monitoring.tf` |
| C5 | caller simulator → the browser's speech vendor | the tester's voice, while they hold the mic button | nothing: the browser sends the audio itself (Chrome to Google's service) | [CONFIRM: the browser vendor's region] | [CONFIRM: the browser vendor's terms] | [CONFIRM: the browser vendor's retention] | `python/pirtis/playground/static/js/caller.js` |

| ID | From → To | Data | Before it crosses | Region | Transfer mechanism (GDPR Ch. V) | Retention | Evidence |
|---|---|---|---|---|---|---|---|
| C6 | off-path reviewer → model vendor | each model turn of a stored call: the call so far as text, the saunas, a 42-day calendar; by default to another model (`REVIEW_MODEL`) | nothing is redacted, as C2; it runs only when the owner runs `make review`, never inside a call | as C2 | as C2 | as C2 | `python/pirtis/ops/review.py` |

**What this diagram says plainly:**

- **C2 leaves the EEA on the OpenAI path.** The data is synthetic customers plus whatever a tester types, which is why testers call as seed personas and never as themselves. Pointing `LLM_BASE_URL` at Azure OpenAI in an EU region keeps C2 inside the EEA.
- **C5 is the browser's doing,** not ours, and a tester avoids it by typing.

## Privacy threats (a LINDDUN pass over this diagram)

| Threat | Where | Finding | Handled by |
|---|---|---|---|
| Linking | turn records | the salted phone hash links one caller's turns across calls, by design for analytics | the salt is a secret; records expire after 90 days |
| Identifying | sessions, feedback | a tester can say their own name or details | seed personas only (scenario cards, the dial page's note); 30-day expiry |
| Data disclosure | C2, C5, C6 | call text leaves the EEA to the model vendor; voice goes to the browser's vendor | synthetic personas; typing as an alternative; the EU model path. Recorded as a known limit (SPEC §12) |
| Unawareness and unintervenability | the caller simulator | the only notice is the dial page's line that calls are kept 30 days; a tester can't delete their own calls | the owner deletes on request (files on the share); a self-service delete is not built |
| Non-compliance | C2 | no DPA with the model vendor is recorded here | [CONFIRM: OpenAI DPA] |

**Not built:**

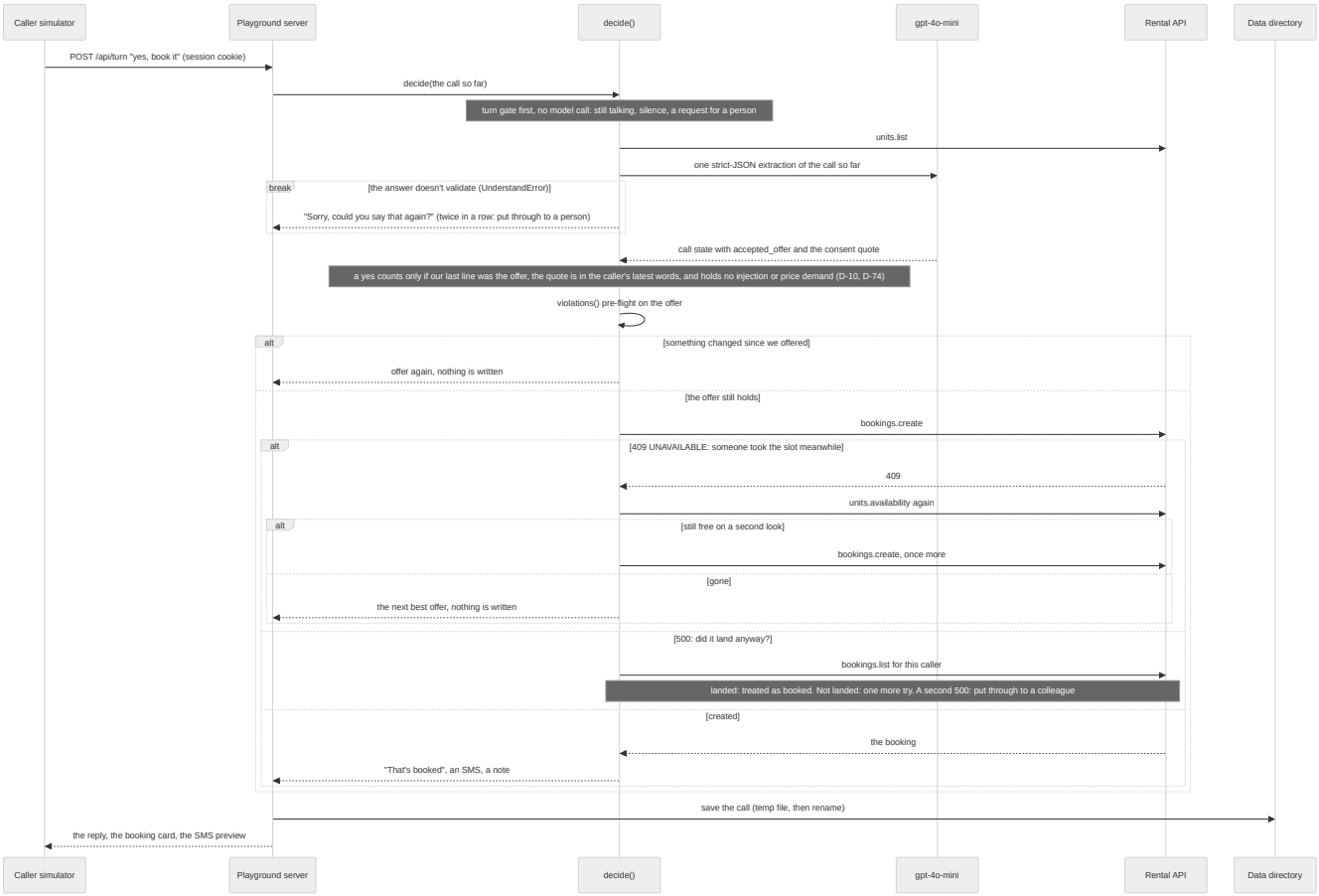
- redaction before C2 (names spoken by the caller go to the model as said)
- a self-service way for a tester to delete their calls
- a privacy notice beyond the dial page's line
- the reviewer on a schedule in Azure (today the owner runs `make review` where the data is)

> Verified against `3e57ec9` on 2026-10-31.

# Sequences

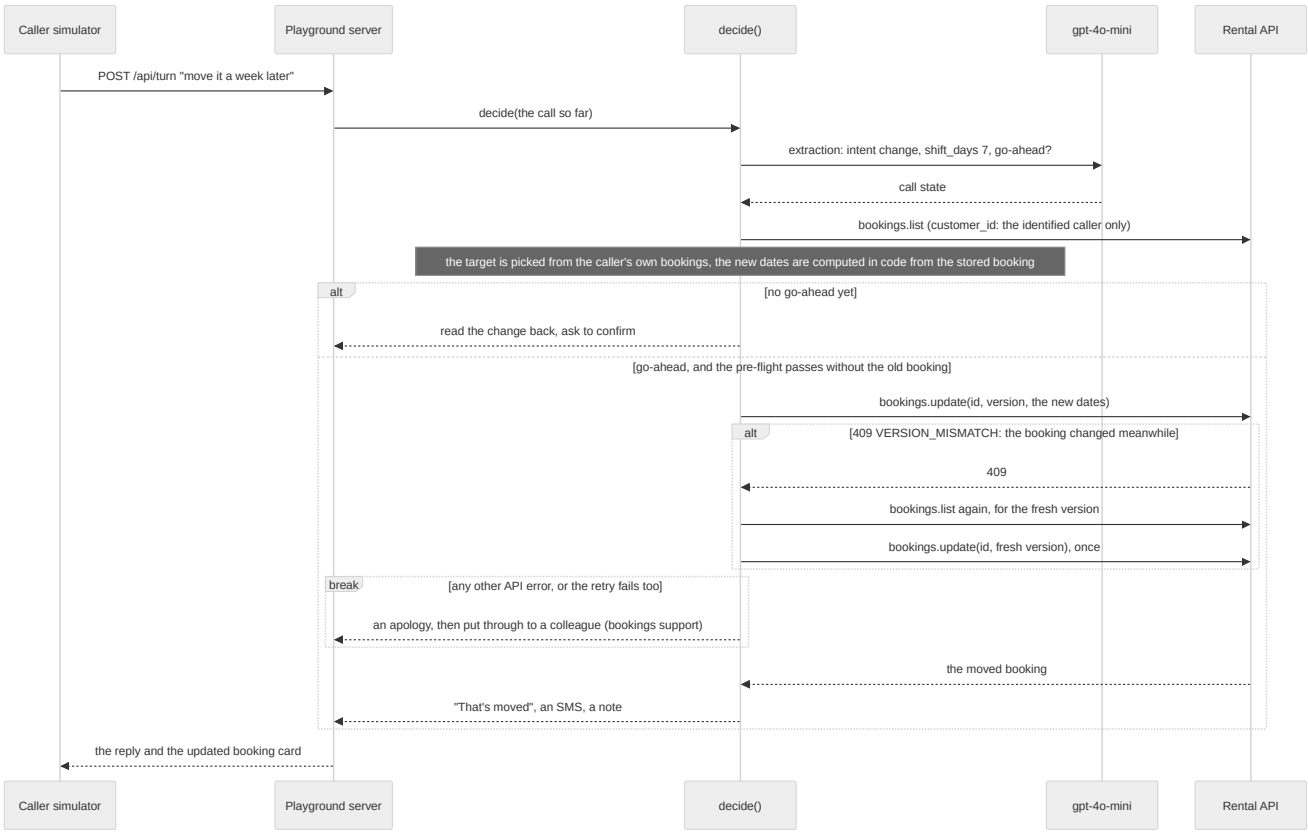
The two paths that write something a customer relies on: a new booking, and a change to one. The participants are the containers in `containers.md`.

## 1 · A booking, on the caller's yes to our read-back




The pre-flight `violations()` mirrors every rule the API enforces and is property-tested against it (`python/pirtis/agent/rules.py`), so a 422 can't happen by construction. The retries are in `create()` (`python/pirtis/agent/policy.py`); a failure no branch handles becomes a hand-off to a person (`python/pirtis/agent/agent.py`).

## 2 · Moving or cancelling a booking




A cancel is the same path with `status: cancelled` as the change. The retry is `update_with_retry()` in `python/pirtis/agent/policy.py`.

**Not built:** an idempotency key on writes. A double submit is safe today only because one call is one session handled one turn at a time, and the 500 branch looks before retrying.

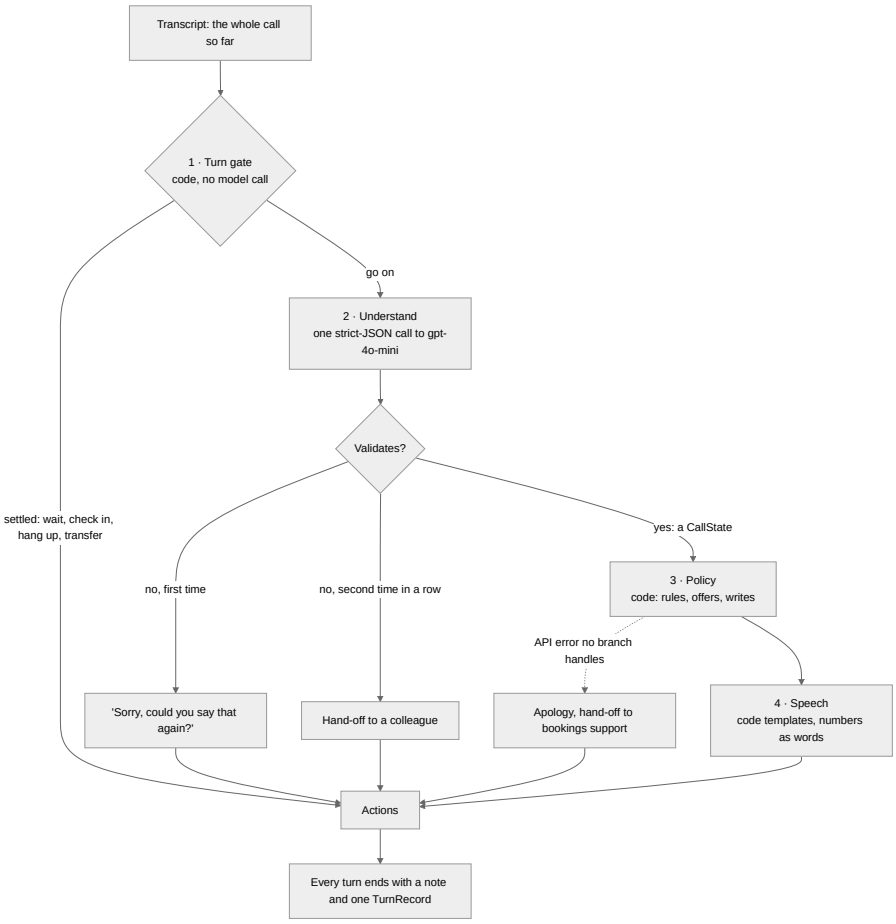
> Verified against `fd7d34f` on 2026-11-01.

# Logic flow

What the receptionist decides on one turn, and in what order. `sequences.md` follows the two writes across the containers; this page follows the decisions inside `decide()`. Every box names the code that does it; `D-nn` points to the decision log.

The agent is **stateless**: each turn it receives the whole call so far and rebuilds what it knows from it. The kit calls `decide()` once per caller turn, so a conversation of any length is this flow, run again on a longer transcript.

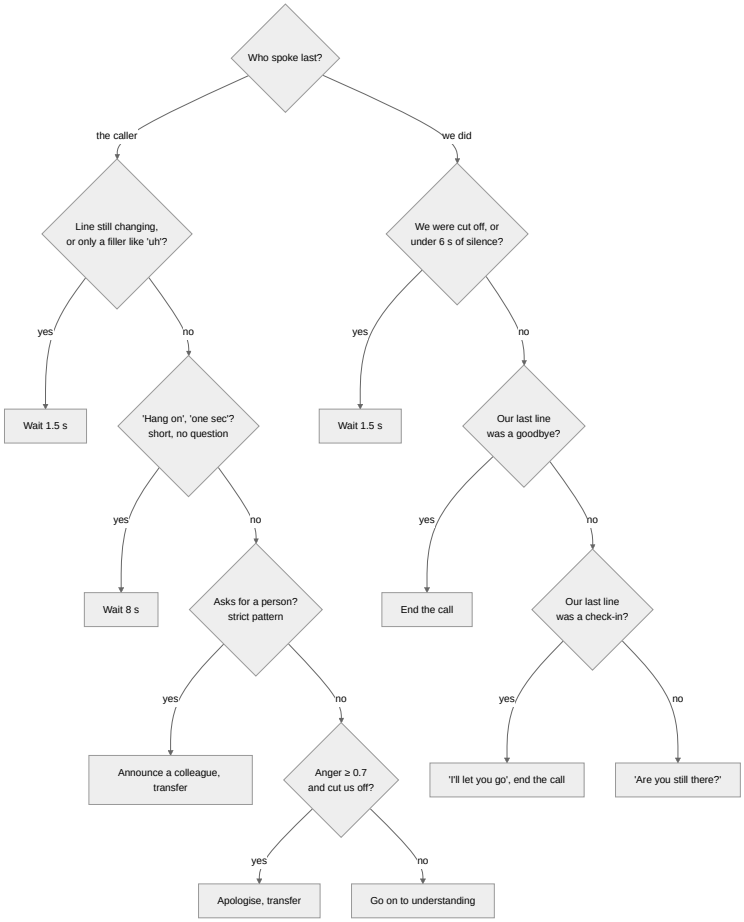
## 1 · One turn, end to end




| Step | Code | Model calls |
|---|---|---|
| 1 · Turn gate | `agent/turn.py` `gate()` | 0 |
| 2 · Understand | `agent/understand.py` `extract()` | 1 |
| 3 · Policy | `agent/policy.py` `next_step()` | 0 |
| 4 · Speech | `agent/speech.py` `render()`, then the `say` handler's digit guard in `agent/actions.py` | 0 |
| Wiring, fallbacks | `agent/agent.py` `decide()`, `_turn()`: never raises (D-19) | — |

## 2 · The turn gate

Runs before anything else, because speaking while the caller is still talking is the one mistake a phone receptionist can't take back (D-04 to D-06).




A paraphrase the patterns miss ("could a human help me?") still reaches the model, whose intents include `human`, `hold` and `goodbye`: a miss costs a model call, not a wrong answer.

## 3 · Understanding

One call, temperature 0, with a strict JSON schema generated from the `CallState` class (D-33, D-49). The prompt holds the rules of reading, the sauna catalogue from the API, a 42-day calendar computed in code (D-16), and the call so far, both sides.

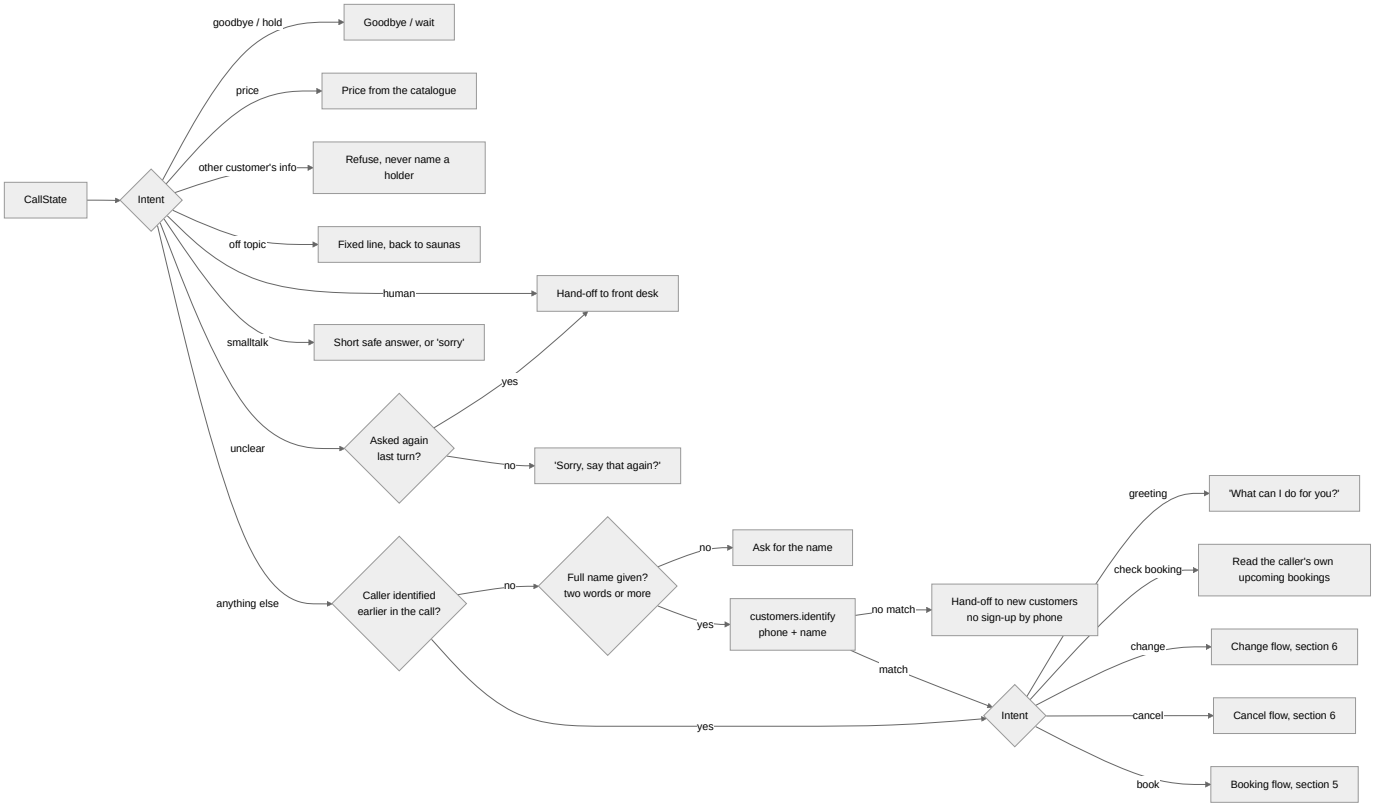
**The model fills, in this order:** a gist; the intent; the caller's name; dates; people; each fact's quote **before** the fact (tow, sauna, heater reason, consent); power; the consent flags last. Evidence before value is what fixed the asthma case (D-69).

**Code then checks what came back** (`CallState._facts_need_the_callers_words`):

| Fact | Counts only if | Otherwise | Decision |
|---|---|---|---|
| Towing capacity | its quote is in the caller's own words and carries a number | dropped: the tow is asked | D-08, D-54 |
| Sauna | its quote is the caller's and names or describes a sauna; code resolves the sauna from the quote if the model left it out | dropped | D-54, D-75 |
| People | always; a hedge ("eight, maybe nine") is raised to the higher number | — | D-79 |
| A "yes" to an offer | its quote is from the caller's latest turn, our last line asked to book, and the turn holds no price demand or instruction to the system | void: the offer is read back again | D-10, D-54, D-74 |
| Name | a name the caller asks about is not theirs | dropped | D-78 |
| Dates, numbers, the sauna id | a real date; 1–50 people; 100–20,000 kg; a sauna in the catalogue | unknown, so it is asked again | D-49 |
| The answer as a whole | valid JSON in the schema | "Sorry, could you say that again?"; twice in a row, a colleague | D-19 |

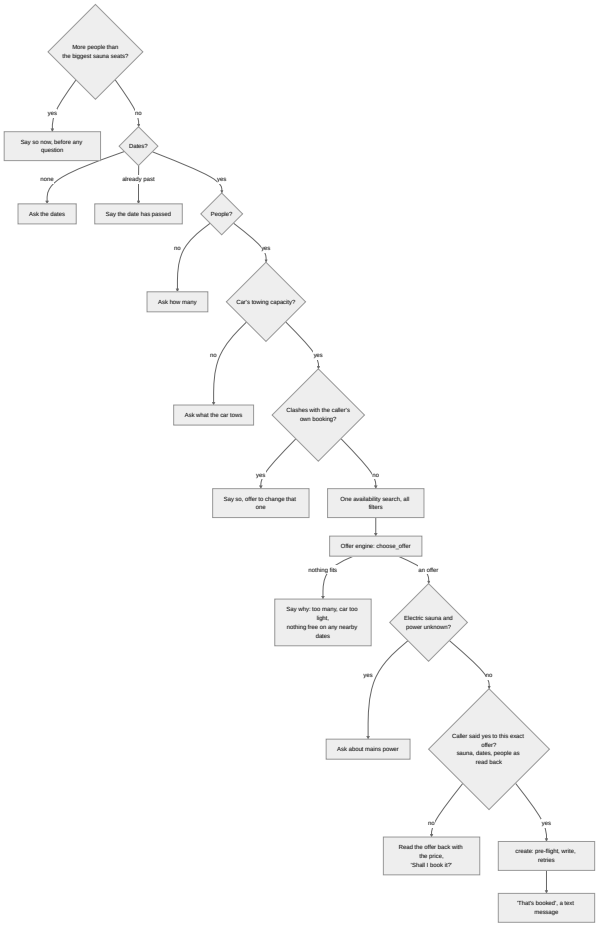
## 4 · The policy

`next_step()` turns the checked state into one decision. Some intents need nobody's data; everything else waits for identification (D-07, D-11).




## 5 · Booking: one question at a time, then an offer

`book()` asks for the **one** next missing fact (D-09), says a problem as soon as it is known (D-68, D-71), and writes only on a yes to exactly what the caller heard (D-10, D-52).



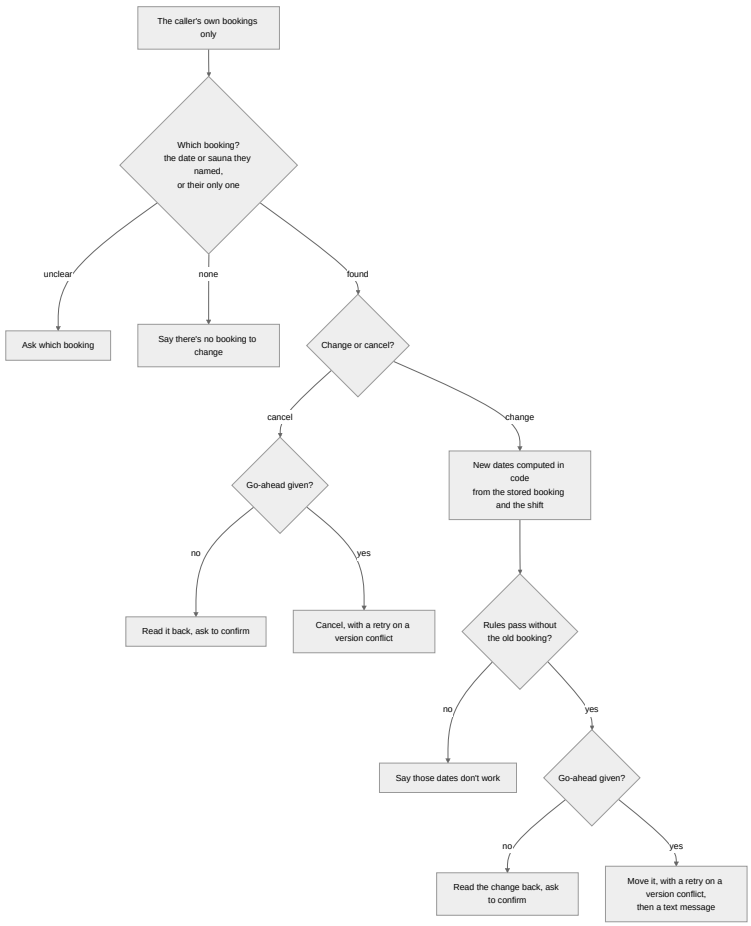
**The offer engine** (`choose_offer()`, D-12, D-35):

1. Drop every sauna that breaks a hard limit: too small, too heavy for the car, electric without power. Hard limits are never overridden.
2. A heater preference **sorts**; it **filters** only when the caller gave a reason ("the smoke bothers me") or named that sauna as a must-have, and a must-have is never filtered out (D-69).
3. A booking touching a weekend is stretched to two days (D-13).
4. The sauna the caller asked for, on their dates, if the rules allow it. If its dates are taken: another sauna on those dates, or, if they insist on that one or nothing else fits, the same sauna moved in time.
5. Otherwise the smallest fitting sauna on their dates.
6. Otherwise the nearest dates that work for any fitting sauna, searching 0, +1, −1, +2, −2 … days, never before today.

Every candidate passes `violations()`, the copy of the API's write rules, so nothing is offered that the API would reject (D-31).

**Writing** (`create()`): the rules run once more before the write. If something changed since the offer, the new offer is read back and nothing is written. A 409 (someone took the slot) means a second look and one more try, or the next best offer; a 500 means checking whether the booking landed before trying once more (D-19). The paths are drawn in `sequences.md`.

## 6 · Changing or cancelling




The booking's own days, and its own turnaround day when the day beyond is free, count as free when it moves (D-15).

## 7 · Speech

`render(decision, voice)` turns each decision into actions from code templates (D-17):

- **one question** per turn, short sentences, offers ending "Shall I book it?"
- **numbers, dates and euros as words**; the `say` handler turns any digit left into words, whatever wrote the line
- **things that aren't done are said outright** ("I can't look at your bookings until I know who's calling"), so no reply can be read two ways (D-51, D-87)
- a **calmer, slower voice** when the caller's latest turn sounds angry or upset

- a **text message** after a booking or a move; a **summary for the colleague** on every transfer

## Where the model's judgement ends

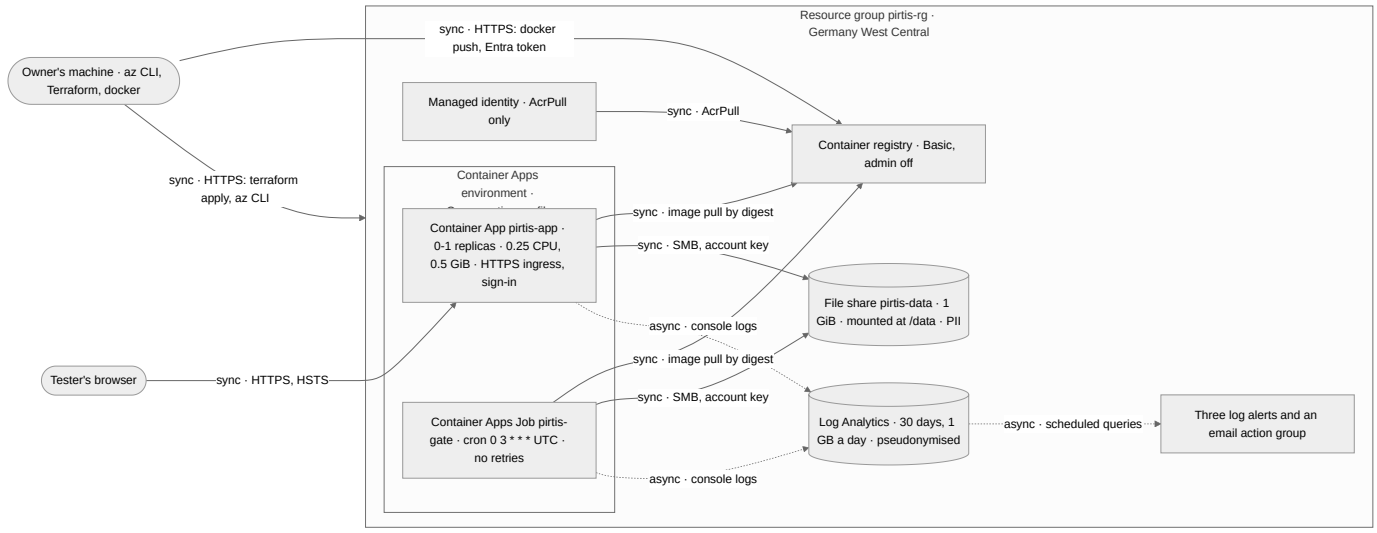
| Decided by the model | Decided by code |
|---|---|
| what the caller wants (the intent) | whether to answer at all this turn |
| what they said: name, dates, people, tow, sauna, heater, power, consent, with quotes | whether each of those facts counts |
| — | which question comes next |
| — | which sauna, which dates, what price |
| — | whether to write, and what |
| — | every word the receptionist says |

So a model mistake can cost an extra question or a "say that again", never a booking that breaks a rule or a reply the code didn't write.

> Verified against `3e57ec9` on 2026-10-31.

# Deployment

Where the containers run. This view is about infrastructure; what talks to what is in `containers.md`. Commands, costs and the bootstrap order are in `deploy/README.md`.




| Resource | Settings that matter | Evidence |
|---|---|---|
| Container App `pirtis-app` | 0 to 1 replicas; 0.25 CPU, 0.5 GiB; external ingress, HTTPS only; probes on `/api/health`; image by digest; secrets: model key, hash salt, session key, accounts | `deploy/terraform/main.tf` |
| Container Apps Job `pirtis-gate` | cron `0 3 * * *` UTC; 30 min timeout; no retries (a failed gate is read, never re-run); `gate-job --suites nightly` | `deploy/terraform/main.tf` |
| Container registry | Basic; admin account off; pulled only by the managed identity | `deploy/terraform/main.tf` |
| Managed identity | one role: AcrPull on the registry | `deploy/terraform/main.tf` |
| Storage account and share | LRS; TLS 1.2; HTTPS only; no public blob access; one 1 GiB share, linked to the environment with the account key | `deploy/terraform/main.tf`, `deploy/Makefile` |
| Log Analytics | 30 days; a 1 GB daily cap | `deploy/terraform/main.tf` |
| Alerts | gate regression, agent errors, slow turns; every 15 minutes over the last hour; email to the owner | `deploy/terraform/monitoring.tf` |
| The image | two stages, bases pinned by digest; uid 10001, read-only code; refuses to start without sign-in accounts | `deploy/Dockerfile`, `.dockerignore` |

**Secrets** are set by the az CLI, never written in these files: the model key, the hash salt, the session key, the sign-in accounts, and the storage key for the share's link. Terraform's state holds the storage keys and, once refreshed, the app's secrets. So the state stays on the owner's machine, owner-only and out of git (`deploy/Makefile`).

**Not built:**

- remote, encrypted Terraform state
- Key Vault references for the secrets
- a managed-identity mount for the share (in preview; the account key is used today)
- private networking: the app has a public endpoint, behind its sign-in
- CI that builds, scans and deploys on every push